

Basic Object-Oriented Concepts

By Edward V. Berard (The Object Agency, Inc.)

- [How People Understand Technology](#)
- [How People Approach Object-Oriented Technology](#)
- [Object-Oriented Terms and Concepts](#)
 - [Objects](#)
 - [Classes, Metaclasses, Parameterized Classes, and Exemplars](#)
 - [Black Boxes and Interfaces](#)
 - [Aggregation](#)
 - [Specialization and Inheritance](#)
 - [Abstract Classes](#)
 - [Operations](#)
 - [Constants](#)
 - [Exceptions](#)
 - [Object Coupling and Object Cohesion](#)
 - [Systems of Objects](#)

How People Understand Technology

There is an old story of how several blind men set out to understand what an elephant was by examining a live specimen. Each of them explored a different part of the elephant's body. One blind man, falling against the elephant's side, proclaimed that an elephant must be very much like a wall. Another, grasping the elephant's ear, decided that an elephant must closely resemble a leaf. One grabbed the elephant's tail and determined that elephants must resemble ropes. Yet another felt the elephant's leg and came away convinced that elephants are very much like trees. Still another held the elephant's trunk and exclaimed that elephants had much in common with snakes.



While there was at least a grain of truth in each blind man's observation, none of them had a complete and accurate understanding of just what an elephant is. We can imagine the many animated debates that these blind men had when they compared notes -- each sure that the others must be wrong.

Often, people studying technology are like the blind men in our story. It is very common to focus intently on one facet of one aspect of a technology while ignoring the vast breadth of the same technology. There is nothing wrong with this -- unless a person studying the facet begins to think that he or she has somehow acquired a good grasp of the entire technology.

To acquire an in-depth understanding of a technology requires a great deal of research. The sheer bulk of material that must be examined is daunting. The task is further complicated by the unfortunate errors

contained in many discussions. Even if an individual can assimilate a large quantity of material and easily identify any errors, there is still the process of analyzing what has been assimilated. To truly understand a technology, an individual must be able to recognize valid patterns, and to easily spot invalid patterns.

How People Approach Object-Oriented Technology

Object-oriented technology is both immense and far-reaching. End users of computer systems and computer-based systems notice the effects of object-oriented technology in the form of increasingly easy-to-use software applications and operating systems and in more flexible services being provided by such industries as banking, telecommunications, and cable television. For the software engineer, object-oriented technology encompasses object-oriented programming languages, object-oriented development methodologies, management of object-oriented projects, object-oriented computer hardware, and object-oriented computer aided software engineering, among others.

It is not surprising, therefore, that there is some confusion regarding object-oriented terms and concepts. In this article, we will provide the reader with working definitions for object-oriented terms and concepts that are necessary for a reader to acquire a basic understanding of object-oriented technology.

Many of the terms commonly used in object-oriented technology were originally used to describe object-oriented programming (coding) concepts. Specifically, although the terms were borrowed from a non-computer-software perspective, they were first used extensively to describe concepts embodied in object-oriented programming languages, such as Smalltalk, C++, and Eiffel. However, these terms are quite useful even if one never intends to write any software at all.

For example, an industrial modeler could create an object-oriented model of a plastics manufacturing facility. Molding machines, plastic parts, and even the "recipes" (proportional combinations) of the chemicals used to create the various plastics could all be described in object-oriented terms. Further, dynamic and static relationships among these items could also be described in object-oriented terms.

Finally, keep in mind that there is no one ultimate set of definitions for object-oriented terms and concepts. Depending on who you are talking to, terms and definitions will vary slightly. This is normal; in different parts of the United States, the same breakfast item might be referred to as a pancake, a griddle cake, a flapjack, or a hot cake. Even in technical arenas, this variation in terminology is common. A chemist might use the terms "valance" and "oxidation state" to identify the same concept.

Object-Oriented Terms and Concepts

Objects

Objects are the physical and conceptual things we find in the universe around us. Hardware, software, documents, human beings, and even concepts are all examples of objects. For purposes of modeling his or her company, a chief executive officer could view employees, buildings, divisions, documents, and benefits packages as objects. An automotive engineer would see tires, doors, engines, top speed, and the current fuel level as objects. Atoms, molecules, volumes, and temperatures would all be objects a chemist might consider in creating an object-oriented simulation of a chemical reaction. Finally, a software engineer would consider stacks, queues, windows, and check boxes as objects.

Objects are thought of as having state. The **state** of an object is the condition of the object, or a set of circumstances describing the object. It is not uncommon to hear people talk about the "state

information" associated with a particular object. For example, the state of a bank account object would include the current balance, the state of a clock object would be the current time, the state of an electric light bulb would be "on" or "off." For complex objects like a human being or an automobile, a complete description of the state might be very complex. Fortunately, when we use objects to model real world or imagined situations, we typically restrict the possible states of the objects to only those that are relevant to our models.

We also think of the state of an object as something that is internal to an object. For example, if we place a message in a mailbox, the (internal) state of the mailbox object is changed, whereas the (internal) state of the message object remains unchanged.

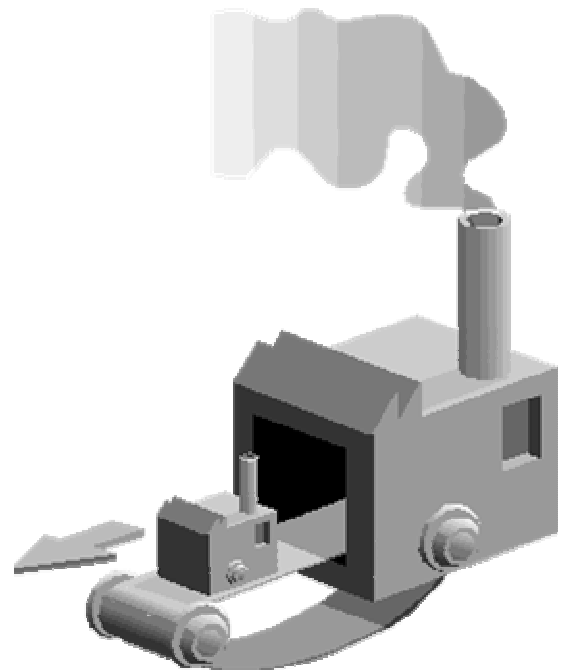
Sometimes people think of objects as being strictly static. That is, the state of an object will not change unless something outside of the object requests the object to change its state. Indeed, many objects are passive (static). A list of names does not spontaneously add new names to itself, nor would we expect it to spontaneously delete names from itself.

However, it is possible for some objects to change their own state. If an object is capable of spontaneously changing its own state, we refer to it as an "**object with life**." (Objects with life are sometimes also called "active objects" or "actors.") Clocks and timers are common examples of objects with life. If we were modeling a business process, we would recognize that salespeople and customers were also objects with life.

Classes, Metaclasses, Parameterized Classes, and Exemplars

There are two broad categories of objects: classes and instances. Users of object-oriented technology usually think of classes as containing the information necessary to create instances, i.e., the structure and capabilities of an instance is determined by its corresponding class. There are three commonly used (and different) views on the definition for "class":

- A **class** is a pattern, template, or blueprint for a category of structurally identical items. The items created using the class are called **instances**. This is often referred to as the "class as a `cookie cutter'" view. As you might guess, the instances are the "cookies."
- A **class** is a thing that consists of both a pattern and a mechanism for creating items based on that pattern. This is the "class as an `instance factory'" view; **instances** are the individual items that are "manufactured" (created) using the class's creation mechanism.
- A **class** is the set of all items created using a specific pattern. Said another way, the class is the set of all **instances** of that pattern.



In this article, we will use the definition of a "class as an `instance factory.'"

We should note that it is possible for an instance of a class to also be a class. A **metaclass** is a class whose instances themselves are classes. This means when we use the instance creation mechanism in a metaclass, the instance created will itself be a class. The instance creation mechanism of this class can, in turn, be used to create instances -- although these instances may or may not themselves be classes.

A concept very similar to the metaclass is the parameterized class. A **parameterized class** is a template for a class wherein specific items have been identified as being required to create non-parameterized classes based on the template. In effect, a parameterized class can be viewed as a "fill in the blanks" version of a class. One cannot directly use the instance creation mechanism of a parameterized class. First, we must supply the required parameters, resulting in the creation of a non-parameterized class. Once we have a non-parameterized class, we can use its creation mechanisms to create instances.

In this article, we will use the term "class" to mean metaclass, parameterized class, or a class that is neither a metaclass nor a parameterized class. We will make a distinction only when it is necessary to do so. Further, we will occasionally refer to "non-class instances." A **non-class instance** is an instance of a class, but is itself not a class. An instance of a metaclass, for example, would *not* be a non-class instance.

In this article, we will sometimes refer to "instantiation." **Instantiation** has two common meanings:

- as a *verb*, instantiation is the process of creating an instance of a class, and
- as a *noun*, an instantiation is an instance of a class.

Some people restrict the use of the term "object" to instances of classes. For these people, classes are not objects. However, when these people are confronted with the concepts of metaclasses and parameterized classes, they have a difficulty attempting to resolve the "problems" these concepts introduce. For example, is a class that is an instance of a metaclass an object -- even though it is itself a class? In this article, we will use the term "object" to refer to both classes and their instances. We will only distinguish between the two when needed.

Black Boxes and Interfaces

Objects are "black boxes." Specifically, the underlying implementations of objects are hidden from those that use the object. In object-oriented systems, it is only the producer (creator, designer, or builder) of an object that knows the details about the internal construction of that object. The consumers (users) of an object are denied knowledge of the inner workings of the object, and must deal with an object via one of its three distinct interfaces:

- the "public" interface. This is the interface that is open (visible) to everybody.
- the "inheritance" interface. This is the interface that is accessible only by direct specializations of the object. (We will discuss inheritance and specialization later in this chapter.) In class-based object-oriented systems, only classes can provide an inheritance interface.
- the "parameter" interface. In the case of parameterized classes, the parameter interface defines the parameters that must be supplied to create an instance of the parameterized class.

Another way of saying that an item is in the public interface of an object is to say that the object "exports" that item. Similarly, when an object requires information from outside of itself (e.g., as with the parameters in a parameterized class), we can say that the object needs to "import" that information.

Aggregation

It is, of course, possible for objects to be composed of other objects. **Aggregation** is either:

- the process of creating a new object from two or more other objects, or
- an object that is composed of two or more other objects.

For example, a date object could be fashioned from a month object, a day object, and a year object. A list of names object, for example, can be thought of as containing many name objects.

A **monolithic object** is an object that has no externally-discernible structure. Said another way, a monolithic object does not appear to have been constructed from two or more other objects. Specifically, a monolithic object can only be treated as a cohesive whole. Those outside of a monolithic object cannot directly interact with any (real or imagined) objects within the monolithic object. A radio button in a graphical user interface (GUI) is an example of a monolithic object.

Composite objects are objects that have an externally-discernible structure, and the structure can be addressed via the public interface of the composite object. The objects that comprise a composite object are referred to as **component objects**. Composite objects meet one or both of the following criteria:

- the state of a composite object is directly affected by the presence or absence of one or more of its component objects, and/or
- the component objects can be directly referenced via the public interface of their corresponding composite object.

It is useful to divide composite objects into two subcategories: heterogeneous composite objects and homogeneous composite objects:

- A **heterogeneous composite object** is a composite object that is conceptually composed of component objects that are not all conceptually the same. For example, a date (made up of a month object, a day object, and a year object) is a heterogeneous composite object.
- A **homogeneous composite object** is a composite object that is conceptually composed of component objects that are all conceptually the same. For example, a list of addresses is a homogeneous composite object.

The rules for designing heterogeneous composite objects are different from the rules for designing homogeneous composite objects.

Specialization and Inheritance

Aggregation is not the only way in which two objects can be related. One object can be a specialization of another object. **Specialization** is either:

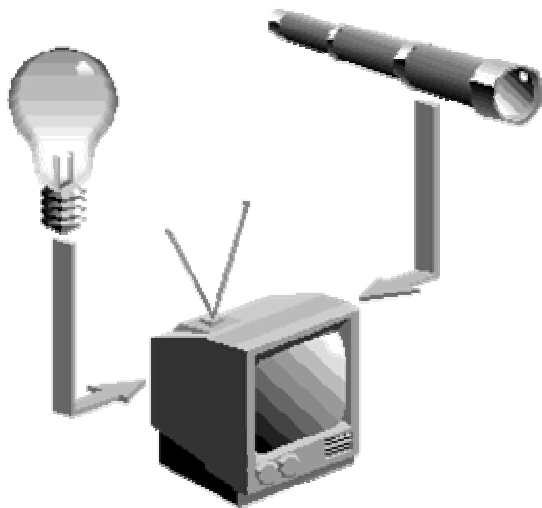
- the process of defining a new object based on a (typically) more narrow definition of an existing object, or
- an object that is directly related to, and more narrowly defined than, another object.

Specialization is usually associated with classes. It is usually only in the so-called "classless" object-oriented systems that we think of specialization for objects other than classes.

Depending on their technical background, there are a number of different ways in which people express specialization. For example, those who are familiar with an object-oriented programming language called Smalltalk refer to specializations as "subclasses" and to the corresponding **generalizations** of these specializations as "superclasses." Those with a background in the C++ programming language use the term "derived class" for specialization and "base class" for corresponding generalizations.

It is common to say that everything that is true for a generalization is also true for its corresponding specialization. We can, for example, define "checking accounts" and "savings accounts" as specializations of "bank accounts." Another way of saying this is that a checking account is a kind of bank account, and a savings account is a kind of bank account. Still another way of expressing this idea is to say that everything that was true for the bank account is also true for the savings account and the checking account.

In an object-oriented context, we speak of specializations as "inheriting" characteristics from their corresponding generalizations. **Inheritance** can be defined as the process whereby one object acquires (gets, receives) characteristics from one or more other objects. Some object-oriented systems permit only **single inheritance**, a situation in which a specialization may only acquire characteristics from a single generalization. Many object-oriented systems, however, allow for **multiple inheritance**, a situation in which a specialization may acquire characteristics from two or more corresponding generalizations.



Our previous discussion of the bank account, checking account, and savings account was an example of single inheritance. A telescope and a television set are both specializations of "device that enables one to see things far away." A television set is also a kind of "electronic device." You might say that a television set acquires characteristics from two different generalizations, "device that enables one to see things far away" and "electronic device." Therefore, a television set is a product of multiple inheritance.

Abstract Classes

We usually think of classes as being complete definitions. However, there are situations where incomplete definitions are useful, and classes that represent these incomplete definitions are equally useful. For example, in everyday conversation, we might talk about such items as bank accounts, insurance policies, and houses. In object-oriented thinking, we often isolate useful, but incomplete, concepts such as these into their own special classes.

Abstract classes are classes that embody coherent and cohesive, but incomplete, concepts, and in turn, make these characteristics available to their specializations via inheritance. People sometimes use the terms "partial type" and "abstract superclass" as synonyms for abstract class. While we would never

create instances of abstract classes, we most certainly would make their individual characteristics available to more specialized classes via inheritance.

For example, consider the concept of an automobile. On one hand, most people know what an automobile is. On the other hand, "automobile" is not a complete definition for any vehicle. It would be quite accurate to describe "automobile" as the set of characteristics that make a thing an automobile, in other words, the "essence of automobile-ness."

Operations

The public interface of an object typically contains three different categories of items:

- operations (sometimes referred to as "method selectors," "method interfaces," "messages," or "methods"),
- constants, and
- exceptions.

An operation in the public interface of an object advertises a functional capability of that object. For example, "deposit" would be an operation in the public interface of a bank account object, "what is current temperature" would be an operation in the public interface of a temperature sensor object, and "increment" would be an operation in the public interface of a counter object.

The actual algorithm for accomplishing an operation is referred to as a **method**. Unlike operations, methods are not in the public interface for an object. Rather, methods are hidden on the inside of an object. So, while users of bank account objects would know that they could make a deposit into a bank account, they would be unaware of the details as to how that deposit actually got credited to the bank account.

We refer to the operations in the public interface of an object as "suffered operations." **Suffered operations** are operations that meet two criteria: they are things that happen to an object, and they are in the public interface of that object. For example, we can say that a bank account "suffers" the operation of having a deposit made into it. The bank account can also "suffer" the operation of being queried as to its current balance. Some people also refer to suffered operations as "exported operations."

There are three broad categories of *suffered* operations, i.e.:

- A **selector** is an operation that tells us something about the state of an object, but cannot, by definition, change the state of the object. An operation that tells us the current balance of a bank account is an example of a selector operation.
- A **constructor** is an operation that has the ability to change the state of an object. For example, an operation in the public interface to a mailbox object that added a message to the mailbox would be a constructor operation. (Please note that some people restrict the definition of the term "constructor" to those operations that cause instances of a class to come into existence.)
- In the context of a homogeneous composite object, an **iterator** is an operation that allows its users to visit (access) each of the component objects that make up the homogeneous composite object. If we have a list of addresses, for example, and we wish to print the entire list, an iterator would

allow us to visit each address object within the list and then, in turn, to print each address.

Iterators can be further divided into two broad categories: active (open) iterators and passive (closed) iterators. Active iterators are objects in their own right. Passive iterators are implemented as operations in the interface of the object over which they allow iteration. Passive iterators are further broken down into selective iterators and constructive iterators. Passive selective iterators do not allow their users to change the object over which the iteration takes place. Passive constructive iterators do allow users to change the object over which iteration takes place.

We can also describe suffered operations as primitive or composite. A **primitive operation** is an operation that cannot be accomplished simply, efficiently, and reliably without direct knowledge of the underlying (hidden) implementation of the object. As an example, we could argue that an operation that added an item to a list object, or an operation that deleted an item from a list object were primitive operations with respect to the list object.

Suppose that we wanted to create a "swap operation," an operation that would swap in a new item in a list, while at the same time swapping out an old item in the same list. This is not a primitive operation since we can accomplish this with a simple combination of the delete operation (deleting the old item) followed by the add operation (adding the new item). The swap operation is an example of a composite operation. A **composite operation** is any operation that is composed, or can be composed, of two or more primitive operations.

Sometimes objects need help in maintaining their characteristics. Suppose, for example, that we wanted to create a "generic ordered list" object. An ordered list is a list that must order its contents from the smallest to the largest. Specifically, every time we add an item to our ordered list, that item would have to be placed in its proper position with respect to all the other items already in the list. By "generic," we mean a template that can be instantiated with the category (class) of items we wish to place in the ordered list.

It would not be unreasonable to implement this object as a parameterized class. Obviously, one of the parameters would be the category of items (e.g., class) that we desired to place in the list. For example, could instantiate (make an instance) the generic ordered list with a "name class" resulting in the creation of an "ordered list of names class."

There is a problem, however. Given that we could instantiate the generic ordered list with just about any category of items, how can we be sure that the ordered lists will know how to properly maintain order -- *no matter what we use to instantiate the generic ordered list*? Suppose, for example, that we wanted an ordered list of "fazoomas." How could the generic list class tell if one fazooma was greater than or less than another fazooma?

A solution would be for the generic ordered list to require a second parameter, a parameter over and above the category of items (class) that we desired to place in the list. This second parameter would be a "<" (less than) operation that worked with the category of items to be placed in the list. In the case of our ordered list of fazoomas, this second parameter would be a "<" that works with fazoomas.

The "<" that worked with fazoomas is an example of a required operation. A **required operation** is an operation that an object needs to maintain its outwardly observable characteristics, but which the object cannot supply itself. Some people refer to required operations as "imported operations."

Constants

In addition to suffered operations, the public interface of an object can also contain constants. **Constants** are objects of constant state. Imagine that we want to create a "bounded list of addresses class." A bounded list is a list that has a fixed maximum number of elements. A bounded list can be empty, and it can contain fewer than the maximum number of elements. It can even contain the maximum number of elements, but it can never contain more than the defined maximum number of elements.

Assume that we place a constant in the public interface of our bounded list of addresses. This constant represents the maximum number of elements that can be placed in the bounded list. Assume also that there is a suffered operation that will tell us how many elements (addresses, in our example) are currently in the bounded list. We can now determine how much room is available in the bounded list by inquiring how many addresses are already in the list, and then subtracting this from the previously-defined constant.

In some cases, as with the bounded list example above, constants are provided more for convenience than necessity. In other cases, such as in the case of encryption algorithms needing a "seed value," constants are an absolute requirement.

Exceptions

A third category of items that can be found in the public interface of objects is exceptions. **Exceptions** have two different definitions:

- an event that causes suspension of normal application execution, and
- a set of information directly relating to the event that caused suspension of normal application execution.

Exceptions can be contrasted with an older, less reliable technology: "error codes." The idea behind error codes was fairly simple. You would request that an application, or part of an application, accomplish some work. One of the pieces of information that would be returned to the requester would be an error code. If all had gone well, the error code would typically have a value of zero. If any problems had occurred, the error code would have a non-zero value. It was also quite common to associate different non-zero values of an error code with specific errors.

Error codes suffered from two major problems:

- No one was forced to actually check the value of returned error codes.
- Changes (additions, deletions, and modifications) in the meanings of the special values assigned to error codes were not automatically passed on to interested parties. Tracking the effects of a changed error code value often consumed a significant amount of resources.

To understand how exceptions directly address both of these issues, we first need to understand how exceptions typically work:

- Exceptions may be defined by the environment or by the user.
- When an exceptional (but not unforeseen) condition occurs, an appropriate exception is activated. (People use different terms to express the activation of an exception. The most common is "raise." Less commonly, people use the terms "throw" or "activate.") This activation may be automatic

(controlled by the environment) or may be expressly requested by the designer of the object or application.

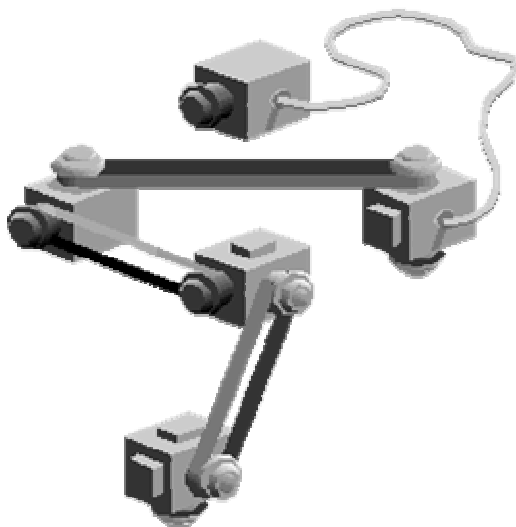
Examples of exceptional conditions include trying to remove something from an empty container, directing an elevator on the top floor to "go up," and attempting to cause a date to take on an invalid value like "February 31, 1993."

- Once the exception is activated, normal application execution stops and control is transferred to a locally defined exception handler, if one is present. If no locally defined exception handler is present or if the exception handler is not equipped to handle the exception, the exception is propagated to the next higher level of the application. *Exceptions cannot be ignored.* An exception will continue to be sent to higher levels of the application until it is either turned off or the application ceases to function.
- An exception handler checks to see what type of exception has been activated. If the exception is one that the handler recognizes, a specific set of actions is taken. Executing a set of actions in response to an exception is known as "handling the exception." Handling an exception deactivates the exception; the exception will not be propagated any further.

Unlike error codes, exceptions cannot be ignored. Once an exception has been activated, it demands attention. In object-oriented systems, exceptions are placed in the public interfaces of objects. Changes in the public interfaces of objects very often require an automatic rechecking of all other objects that invoke operations in the changed objects. Thus, changes in exceptions result in at least a partially automated propagation of change information.

Object Coupling and Object Cohesion

Engineers have known for centuries that the less any one part of a system knows about any other part of that same system, the better the overall system. Systems whose components are highly independent of each other are easier to fix and enhance than systems where there are strong interdependencies among some or all of the components. Highly independent system components are possible when there is minimal coupling among the components, and each component is highly cohesive.

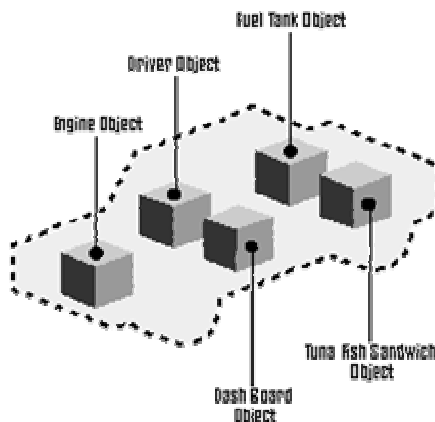


Coupling is a measure of the strength of the connection between any two system components. The more any one component knows about another component, the tighter (worse) the coupling is between those two components. Cohesion is a measure of how logically related the parts of an individual component are to each other, and to the overall component. The more logically related the parts of a component are to each other the higher (better) the cohesion of that component.

The objects that make up an object-oriented system exhibit object coupling and object cohesion. **Object**

coupling describes the degree of interrelationships among the objects that make up a system. The more any one object knows about any other object in the system, the tighter (worse) the coupling is between those two objects.

To construct systems from objects, we must couple (to some degree) the objects that comprise the system. This is *necessary* object coupling. However, if in the design of an individual object, we give that object direct knowledge of other specific objects, we are unnecessarily coupling the objects. *Unnecessary* object coupling reduces both the reusability of individual objects, and the reliability of the systems that contain unnecessarily coupled objects.



Object cohesion, on the other hand, is a measure of how logically related the components of the *external* view of an object are to each other. For example, if we are told that a date object is comprised of a month object, a day object, a year object, and the color blue, we should recognize that the color blue is not appropriate, and lowers the cohesion of the date object. We want our objects to be as cohesive as possible for two reasons. First, objects with low cohesion are more likely to be changed, and are more likely to have undesirable side effects when they are changed. Second, objects with low cohesion are seldom easily reusable.

Systems of Objects

In constructing object-oriented models and object-oriented applications, one quickly finds that single classes and single instances are not enough. You need some way of creating and dealing with large objects. A **system of objects** is defined as two or more interacting or interrelated, non-nested objects. (We exclude simple aggregations of composite objects from our definition of systems of objects.)

Systems of objects fall into two general categories:

- **kits**, which are collections of items (classes, metaclasses, parameterized classes, non-class instances, other kits, and/or systems of interacting objects) all of which support a single, large, coherent, object-oriented concept, such as computer graphics windows or insurance policies. There may indeed be some physical connection among some of the members of a given kit. However, kits are "granular." While all the components of a kit are logically related, there are very few physical connections that bind them together.
- **systems of interacting objects**, which are collections of items (classes, metaclasses, parameterized classes, non-class instances, kits, and/or other systems of interacting objects) all of which support a single, large, coherent, object-oriented concept, and in which there must be a direct or indirect physical connection between any two arbitrary objects within the collection. Further, systems of interacting objects have at least one internal, independently executing thread of control. Lastly, systems of interacting objects may exhibit multiple, completely disjoint public interfaces.

Kits resemble libraries. Say, for example, that we had to create a computer application with a graphical user interface. Graphical user interfaces normally contain several different types of windows. It would be very useful if we had a library of windows and window components from which we could construct

any window we desired. Windows are objects, and the components of windows (buttons and check boxes) are themselves objects. A collection of windows and window components can be viewed as a kit.

Systems of interacting objects, on the other hand, resemble applications. For example, suppose that we wanted to construct an object-oriented application that controlled the elevators in a particular building. We would assemble elevators, buttons, lamps, panels, and other objects into a working application that would control the elevators. Such an application would not be viewed as a library, but as a highly cohesive whole. The elevator controller application is a system of interacting objects.

